

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.(BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 3

Duration : 1 Hour
Total Marks:30

Annexure A

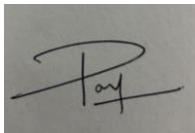
Constraint-Based Problem Solving

Code of Conduct:

I (PAVAN.P / 192411017) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A

Constraint-Based Problem Solving

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

Constraints:

- i. D1 cannot work Night shift
- ii. D2 must work before D3 (Morning < Afternoon < Night)
- iii. Only one doctor per shift
- iv. D3 cannot work Morning
- v. All doctors must be assigned exactly one shift

Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

Constraints:

- i. Movement allowed: Up, Down, Left, Right
- ii. Cost of each move = 1
- iii. Cannot pass through obstacles
- iv. Use Manhattan Distance heuristic

Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost.

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right
- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:

- a) Analyze all the given constraints and explain how they affect the robot's decision-making process.
- b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.
- c) Demonstrate how the robot applies AI concepts such as:
 - Intelligent agents
 - Rationality
 - Environment type

d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

1. ANSWER: -

Doctors:

- D1
- D2
- D3

Shifts:

- Morning (M)
- Afternoon (A)
- Night (N)

Constraints:

1. D1 cannot work **Night**.
2. D2 must work **before** D3 (Morning < Afternoon < Night).
3. Only **one doctor per shift**.
4. D3 cannot work **Morning**.
5. Every doctor must be assigned exactly **one shift**.

Step 1: Assign D1

Possible shifts for D1:

- Morning
- Afternoon
- Night

Choose **D1 = Morning**.

Current Assignment:

- D1 → Morning

Step 2: Assign D2

Remaining shifts:

- Afternoon
- Night

Try **D2 = Afternoon**.

Current Assignment:

- D1 → Morning
- D2 → Afternoon

Step 3: Assign D3

Remaining shift:

- Night

Check constraints:

- $D3 \neq \text{Morning}$
- $D2 \text{ before } D3 \rightarrow \text{Afternoon before Night}$
- One doctor per shift

All constraints satisfied.

Solution Found

2. ANSWER: -

Problem Statement

A robot operates in a **5 × 5 grid** and must move from the **Start (S)** to the **Goal (G)**.

Constraints

- Movement allowed: **Up, Down, Left, Right**
- Cost of each move = **1**
- Robot cannot move through obstacles.
- Manhattan Distance heuristic is given.
- Use **Breadth-First Search (BFS)** to find the shortest path.

Where:

- **S** = Start
- **G** = Goal
- **X** = Obstacle
- **.** = Free Cell

Algorithm Used – Breadth-First Search (BFS)

BFS is an uninformed search algorithm that explores all neighbouring nodes level by level. It uses a **FIFO (First-In, First-Out) Queue** and guarantees the shortest path when all edge costs are equal.

Step 1: Initial State

Queue:

[(1,1)]

Visited:

(1,1)

Step 2: Expand (1,1)

Possible moves:

- Right (1,2)
- Down (2,1)

Queue:

[(1,2), (2,1)]

Step 3: Expand (1,2)

Possible moves:

- Right (1,3)

Queue:

[(2,1), (1,3)]

Step 4: Expand (2,1)

Possible moves:

- Down (3,1)

Queue:

[(1,3), (3,1)]

Step 5: Expand (1,3)

Possible moves:

- Down (2,3)

Queue:

[(3,1), (2,3)]

Step 6: Expand (3,1)

Possible moves:

- Down (4,1)

Queue:

[(2,3), (4,1)]

Step 7: Expand (2,3)

Possible moves:

- Down (3,3)

Queue:

[(4,1), (3,3)]

Step 8: Expand (4,1)

Possible moves:

- Right (4,2)

Queue:

[(3,3), (4,2)]

Step 9: Expand (3,3)

Possible moves:

- Right (3,4)

Queue:

[(4,2), (3,4)]

Step 10: Expand (4,2)

Possible moves:

- Down (5,2)

Queue:

[(3,4), (5,2)]

Step 11: Expand (3,4)

Possible moves:

- Right (3,5)

Queue:

[(5,2), (3,5)]

Step 12: Expand (5,2)

Possible moves:

- Right (5,3)

Queue:

[(3,5), (5,3)]

Step 13: Expand (3,5)

Possible moves:

- Down (4,5)

Queue:

[(5,3), (4,5)]

Step 14: Expand (5,3)

Possible moves:

- Right (5,4)

Queue:

[(4,5), (5,4)]

Step 15: Expand (4,5)

Possible moves:

- Down (5,5) **Goal Reached**

Queue:

[(5,4), (5,5)]

Node Expansion Order

(1,1)
(1,2)
(2,1)
(1,3)
(3,1)
(2,3)
(4,1)
(3,3)
(4,2)
(3,4)
(5,2)
(3,5)
(5,3)
(4,5)
(5,5)

Optimal Path

S Right Right Down Down Right Right Down Down G

Cost Calculation

Each move costs 1.

Number of moves = 8

Total Cost = 8

Manhattan Distance Heuristic

The Manhattan Distance is calculated as:

$$h(n)=|x_{goal}-x|+|y_{goal}-y|$$

BFS does **not** use the Manhattan Distance heuristic or a priority queue. The Manhattan Distance heuristic is used by the **A*** search algorithm. In BFS, nodes are expanded using a **FIFO queue**, and because every move has the same cost, BFS guarantees the shortest path.

3.ANSWER : -

a) Constraint Analysis and Impact on Decision-Making

The robot operates under several critical constraints that directly shape its decision-making:

- Grid-based movement (up/down/left/right): Limits the robot to four discrete actions per step. This simplifies planning but prevents diagonal shortcuts, potentially increasing path length.
- Blocked cells (obstacles): The robot must detect and avoid these dynamically. Since the full map is unknown initially, it may waste moves exploring dead ends before discovering blockages.
- Limited sensing (only adjacent cells): The robot cannot precompute a global optimal path. It must explore incrementally, updating its internal map as it moves—this defines an online search scenario.
- Variable movement costs (1 for normal, 3 for risky zones): Introduces a trade-off between safety and efficiency. The robot must weigh whether a shorter but riskier path is worth the extra cost.
- Minimize total cost while ensuring safety: Forces the robot to balance exploration (to find better paths) with exploitation (using known safe routes). Over-exploration risks unnecessary exposure to high-cost zones.
- Dynamic knowledge updates: The robot cannot rely on a one-time plan. It must continuously sense-plan-act, revising its strategy whenever new obstacles or risky zones are discovered.

Together, these constraints define a partially observable, cost-sensitive, sequential decision problem requiring an adaptive, incremental search strategy.

b) Solution Approach: Online Uniform Cost Search (UCS)

Chosen Strategy: Online Uniform Cost Search (UCS) with risk-aware cost function.

Why UCS?

- Guarantees the least-cost path in any known portion of the map.

- Naturally handles variable edge costs (1 vs. 3).
- Works incrementally, making it ideal for partial observability.

Step-by-Step Process:

1. Initialization:
 - a. Start at position S.
 - b. Maintain an internal map (initially only S and its sensed neighbors).
 - c. Use a priority queue (frontier) ordered by cumulative cost $g(n)$.
2. Sensing:
 - a. At each step, sense all adjacent cells (up/down/left/right).
 - b. Label each as free, obstacle, or risky.
3. Expansion:
 - a. Expand the node with the lowest cumulative cost from the frontier.
 - b. Move the robot to that cell.
 - c. Update the internal map with newly sensed neighbors.
4. Cost Assignment:
 - a. Normal move $\rightarrow$ cost = 1
 - b. Risky zone $\rightarrow$ cost = $1 + 2 = 3$
 - c. Obstacle $\rightarrow$ not added to frontier
5. Replanning:
 - a. If a planned path becomes invalid (new obstacle discovered), discard it and re-run UCS from the current position using the updated map.
6. Termination:
 - a. Stop when the robot reaches G (goal/survivor).
 - b. The path taken is guaranteed to be least-cost given the information available at each step.

This approach ensures optimal navigation under uncertainty while adapting to new information in real time.

c) Application of Core AI Concepts (2.5 marks)

1. Intelligent Agent:
The robot is an autonomous intelligent agent that:

- Perceives its environment via sensors (adjacent cell states).
- Acts via movement commands (up/down/left/right).
- Learns by updating its internal map.
- Reasons using search algorithms to select the best next action.

2. Rationality:
The agent is rational because it:

- Maximizes expected utility by minimizing total movement cost.
- Uses all available percepts to make optimal decisions.
- Adapts to new information to maintain rationality over time.

3. Environment Type (PEAS Framework):

Dimension	Description
Partially Observable	Robot only sees adjacent cells; full map unknown initially
Deterministic	Actions have predictable outcomes (move up → cell above, unless blocked)
Sequential	Current action affects future possibilities (path choices matter)
Static	Environment doesn't change unless robot moves (obstacles fixed)
Discrete	Finite set of states (grid cells) and actions (4 directions)
Single-agent	Only one robot operating; no other agents to coordinate with

This places the problem in the partially observable, deterministic, sequential, static, discrete, single-agent category—ideal for online search.

d) Justification of Approach (2 marks)

Online Uniform Cost Search is the most suitable because:

1. Partial Observability:
Handles
Unlike A* or BFS (which assume full map knowledge), online UCS builds the map incrementally and replans as needed—critical when the robot only sees adjacent cells.

2. Guarantees Optimality in Known Subgraphs:
UCS expands nodes in order of increasing path cost, ensuring the first time the goal is reached, it's via the least-cost path given current knowledge.
3. Accommodates Variable Edge Costs:
The risk penalty (+2) is naturally integrated into the cost function, unlike BFS (which assumes uniform cost).
4. Anytime and Interruptible:
The robot can act after each expansion, making it suitable for real-time deployment where computation time is limited.
5. Robust to Dynamic Updates:
When new obstacles are found, the algorithm discards invalid paths and continues from the current state—no need to restart from scratch.

Alternatives like greedy best-first search might be faster but risk suboptimal paths. A* requires a heuristic and full map knowledge, which isn't available here. Thus, online UCS strikes the best balance between optimality, adaptability, and computational feasibility under the given constraints.